

Phase 3 Implementation Report

Operating Systems Fall 2025

Khaleeqa Aasiyah Garrett

Kag9691 | Custom Shell Implementation

Architecture and Design

High-Level System Architecture

Phase 3 extends the **MyShell** system from Phase 2 by transforming the server from a **single-client sequential model** into a **multithreaded concurrent server**.

While Phase 2 enabled remote command execution over TCP/IP, Phase 3 introduces **parallel client handling** through **POSIX threads (pthread)**, allowing multiple clients to connect and execute commands simultaneously without blocking each other.

My Multithreaded Network Shell Architecture consists of:

1. **Multithreaded Server (server executable)**
 - Listens on **PORT 8080** for incoming TCP connections.
 - Spawns a **dedicated thread per client** using `pthread_create()`.
 - Each thread independently handles receiving commands, executing them, and returning output.
 - Maintains thread safety using mutexes for shared resources (client counter, console output).
2. **Client (client executable)**
 - Remains identical to Phase 2, serving as the user interface.
 - Sends commands and displays results but can now connect concurrently with other clients.
3. **Shared Shell Core (Phase 1 Components)**
 - The modular components (`executor.o`, `parser.o`, `memory.o`, `utils.o`) remain unchanged.
 - Linked to the server, providing consistent command parsing and execution logic.

Key Design Decisions

- **Thread-per-Client Model:**
 - Each new connection is handled by a separate thread, allowing multiple clients to execute commands in parallel.
- **Thread-Safe Console Output:**
 - Introduced a global `output_mutex` to prevent interleaved server logs when multiple threads print simultaneously.
- **Client Identification System:**
 - Implemented a global client counter protected by `counter_mutex`, ensuring each client receives a unique ID (Client #1, Client #2, etc.).
- **Lightweight Synchronization:**
 - Only minimal shared data is protected by mutexes, avoiding unnecessary locking and maximizing concurrency.
- **Detached Threads for Automatic Cleanup:**
 - Threads are detached using `pthread_detach()` to prevent resource leaks and eliminate the need for explicit joins.

File Structure and Organization

File	Role	Dependencies
server.c	Multithreaded TCP server with thread creation and client handling	<pthread.h>, <netinet/in.h>, Phase 1 core objects
client.c	User interface and communication logic	Standard C libraries
shell.h	Common headers, constants, structs	Used by both server and client
Makefile	Builds both executables	gcc, pthread, etc.
executor.c, parser.c, etc.	Core shell logic (unchanged from Phase 1/2)	Phase 1 logic

Implementation Highlights

1. Thread Creation and Client Handling

Each time a client connects, the main server thread:

- Accepts the connection via `accept()`
- Allocates a `ClientInfo` structure
- Locks the counter mutex, increments the global client count, and assigns a unique ID
- Creates a new thread to handle that client

Snippet:

```
pthread_t thread_id;
ClientInfo *info = malloc(sizeof(ClientInfo));
info->socket = client_socket;
strcpy(info->ip, inet_ntoa(client_addr.sin_addr));
info->port = ntohs(client_addr.sin_port);

pthread_mutex_lock(&counter_mutex);
info->client_number = ++client_count;

pthread_mutex_unlock(&counter_mutex);

pthread_create(&thread_id, NULL, handle_client_thread, info);
pthread_detach(thread_id);
```

2. Thread-Safe Logging

All server logs are wrapped with a thread-safe printing function:

```
void thread_safe_printf(const char *format, ...) {  
    pthread_mutex_lock(&output_mutex);  
    va_list args;  
    va_start(args, format);  
    vprintf(format, args);  
    va_end(args);  
    pthread_mutex_unlock(&output_mutex);  
}
```

This prevents garbled console output when multiple threads log simultaneously.

3. Worker Thread Logic

Each thread handles a single client independently:

1. Receives command via `recv()`
2. Parses and executes using the existing Phase 1 executor
3. Captures stdout/stderr via `capture_command_output()`
4. Sends results back to client
5. Exits gracefully when client types `exit`

4. Thread-Safe Client Numbering

Protected using a global mutex:

```
pthread_mutex_t counter_mutex = PTHREAD_MUTEX_INITIALIZER;  
int client_count = 0;
```

Execution Instructions

The project is compiled using the provided `Makefile` and executed by running the two separate executables.

1. Compilation

1. Open a terminal in the project directory.
2. Run the make command to build both the server and client:

Bash

```
make
```

This creates two executables: `./server` and `./client`.

2. Running the Server

1. Open a terminal to start the server:

Bash

```
./server
```

The server will initialize and begin listening for client connections

3. Running Multiple Clients

1. Open a different terminal and return to the project directory and run each client in a new directory.

Bash

```
./client
```

Clients can now connect simultaneously and execute commands independently.

Testing

1. Basic Commands

Test Commands:

Client 1: echo one two three
Client 2: ls -l
Client 3: pwd and unknowncmd
Client 4: echo Hello World

Screenshot 1: Basic Commands

The screenshots show a server interface with multiple clients connected. The server logs show the following sequence of events:

- Client 1 (127.0.0.1:59566) sends 'unknowncmd'. The server logs an error: 'Command not found: unknowncmd'.
- Client 3 (127.0.0.1:12124) sends 'pwd'. The server outputs: '/home/kag9691/cshell'.
- Client 4 (127.0.0.1:35796) sends 'echo Hello World'. The server outputs: 'Hello World'.
- Client 3 (127.0.0.1:12124) sends 'echo one two three'. The server outputs: 'one two three'.
- Client 2 (127.0.0.1:35796) sends 'ls -l'. The server outputs a directory listing:

File	Permissions	Owner	Group	Size	Modified
shell.h	-rw-rw-r--	kag9691	kag9691	1616	Nov 7 09:40
utils.c	-rw-rw-r--	kag9691	kag9691	3309	Nov 7 09:40
utils.o	-rw-rw-r--	kag9691	kag9691	2760	Nov 7 09:45

The server also displays a welcome message and a list of updates available for the shell.

Explanation: These tests verify that the multithreaded server can handle commands from multiple clients simultaneously. Each client's thread receives the command, parses it, executes it, and sends the output or error back only to that client.

- Valid commands (echo, pwd, ls -l) produce the correct output sent to the respective client.
- Invalid commands (unknowncmd) are caught, and an error message is returned without affecting other clients.
- The test confirms that thread-safe output and client identification work properly, so commands from different clients don't interfere with each other.

2. Output Redirection (>)

Test Commands:

```
Client 3: $ echo "Test output" > output.txt
```

```
Client 3: $ cat output.txt
```

```
Client 1: $ ls -l > filelist.txt
```

```
Client 1: $ cat filelist.txt
```

Screenshot 2: Output Redirection (>)

```
khalaasiyah — kag9691@DCLAP-V1525-CSD: ~/cshell — ssh kag9691@a...
[RECEIVED] [Client #4 - 127.0.0.1:35796] Received command: "echo Hello World"
[EXECUTING] [Client #4 - 127.0.0.1:35796] Executing command: "echo Hello World"
[OUTPUT] [Client #4 - 127.0.0.1:35796] Sending output to client:
Hello World
[RECEIVED] [Client #3 - 127.0.0.1:12124] Received command: "echo one two three"
[EXECUTING] [Client #3 - 127.0.0.1:12124] Executing command: "echo one two three"
[OUTPUT] [Client #3 - 127.0.0.1:12124] Sending output to client:
one two three
[RECEIVED] [Client #4 - 127.0.0.1:35796] Received command: "exit"
[INFO] [Client #4 - 127.0.0.1:35796] Client requested disconnect. Closing connection.
[INFO] Client #4 disconnected.
[RECEIVED] [Client #3 - 127.0.0.1:12124] Received command: "echo "Test output" > output.txt"
[EXECUTING] [Client #3 - 127.0.0.1:12124] Executing command: "echo "Test output" > output.txt"
[OUTPUT] [Client #3 - 127.0.0.1:12124] Command executed successfully with no output.
[RECEIVED] [Client #3 - 127.0.0.1:12124] Received command: "cat output.txt"
[EXECUTING] [Client #3 - 127.0.0.1:12124] Executing command: "cat output.txt"
[OUTPUT] [Client #3 - 127.0.0.1:12124] Sending output to client:
Test output
[RECEIVED] [Client #1 - 127.0.0.1:59566] Received command: "ls -l > filelist.txt"
[EXECUTING] [Client #1 - 127.0.0.1:59566] Executing command: "ls -l > filelist.txt"
[OUTPUT] [Client #1 - 127.0.0.1:59566] Sending output to client:
total 244
-rwxrwxr-x 1 kag9691 kag9691 16936 Nov 7 09:45 client
-rw-rw-r-- 1 kag9691 kag9691 8508 Nov 7 09:40 client.c
-rw-rw-r-- 1 kag9691 kag9691 5504 Nov 7 09:45 client.o
-rw-rw-r-- 1 kag9691 kag9691 17977 Nov 7 09:40 executor.c
-rw-rw-r-- 1 kag9691 kag9691 5912 Nov 7 09:45 executor.o
-rw-rw-r-- 1 kag9691 kag9691 0 Nov 7 09:53 filelist.txt
-rw-rw-r-- 1 kag9691 kag9691 1785 Nov 7 09:40 input.c
-rw-rw-r-- 1 kag9691 kag9691 2621 Nov 7 09:40 main.c
-rw-rw-r-- 1 kag9691 kag9691 1802 Nov 7 09:42 Makefile
-rw-rw-r-- 1 kag9691 kag9691 2998 Nov 7 09:40 memory.c
-rw-rw-r-- 1 kag9691 kag9691 2016 Nov 7 09:45 memory.o
-rw-rw-r-- 1 kag9691 kag9691 41168 Nov 7 09:40 myshell
-rw-rw-r-- 1 kag9691 kag9691 12 Nov 7 09:52 output.txt
-rw-rw-r-- 1 kag9691 kag9691 14563 Nov 7 09:40 parser.c
-rw-rw-r-- 1 kag9691 kag9691 8216 Nov 7 09:45 parser.o
-rwxrwxr-x 1 kag9691 kag9691 35096 Nov 7 09:45 server
-rw-rw-r-- 1 kag9691 kag9691 18497 Nov 7 09:45 server.c
-rw-rw-r-- 1 kag9691 kag9691 11744 Nov 7 09:45 server.o
-rw-rw-r-- 1 kag9691 kag9691 1616 Nov 7 09:40 shell.h
-rw-rw-r-- 1 kag9691 kag9691 3309 Nov 7 09:40 utils.c
-rw-rw-r-- 1 kag9691 kag9691 2760 Nov 7 09:45 utils.o
$

khalaasiyah — kag9691@DCLAP-V1525-CSD: ~/cshell — ssh kag9691@a...
New release '24.04.3 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

----- Message last updated: 12 May 2022 -----

Welcome to Delis's server
Please report problems to "nyuad.it@nyu.edu"

Last login: Fri Nov 7 09:41:17 2025 from 10.228.235.67
kag9691@DCLAP-V1525-CSD:~$ cd cshell
kag9691@DCLAP-V1525-CSD:~/cshell$ ./client
Connected to remote shell server.
Type 'exit' to quit

$ pwd
/home/kag9691/cshell
$ echo one two three
one two three
$ echo "Test output" > output.txt
$ cat output.txt
Test output
$
```

Explanation: These tests check that commands can redirect output to files instead of sending it to the client terminal. Each client writes to its own file, and cat confirms the output was saved correctly. The test also shows that multiple clients can redirect output at the same time without affecting each other.

3. Input Redirection (<)

Test Commands:

```
Client3: $ echo -e "Line 1\nLine 2\nLine 3" > input.txt
```

```
Client3: $ cat < input.txt
```

```
Client 3: $ echo -e "zebra\napple\nbanana" > words.txt
```

```
Client 3: $ sort < words.txt
```

Screenshot 3: Input Redirection (<)

The image contains four terminal window screenshots from a server named 'khalaasiyah' with IP 'kag9691@DCLAP-V1525-CSD'. The top-left window shows a client sending a file 'input.txt' containing 'Line 1\nLine 2\nLine 3' and the server successfully receiving and displaying it. The top-right window shows a client sending a file 'words.txt' containing 'zebra\napple\nbanana' and the server successfully receiving and displaying it. The bottom-left window shows a client sending a file 'input.txt' containing 'Line 1\nLine 2\nLine 3' and the server successfully receiving and displaying it. The bottom-right window shows a client sending a file 'words.txt' containing 'zebra\napple\nbanana' and the server successfully receiving and displaying it.

Explanation: These tests check that commands can read input from files instead of the client terminal. The server redirects the file content to the command's stdin, and the output is sent back to the client. This ensures that input redirection works correctly for multiple clients without interfering with each other.

4. Error Redirection (2>)

Test Commands:

Client 1: \$ ls nonexistent_file 2> error.txt

Client 1: \$ cat error.txt

Client 2: \$ echo "success" 2> error2.txt

Client 2: \$ cat error2.txt

Screenshot 4: Error Redirection (2>)

```
khalaaasiyah — kag9691@DCLAP-V1525-CSD: ~/cshell — ssh kag9691@a...
apple
banana
zebra
[RECEIVED] [Client #1 - 127.0.0.1:59566] Received command: "ls nonexistent_file
2> error.txt"
[EXECUTING] [Client #1 - 127.0.0.1:59566] Executing command: "ls nonexistent_fil
e 2> error.txt"
[OUTPUT] [Client #1 - 127.0.0.1:59566] Command executed successfully with no out
put.
[RECEIVED] [Client #1 - 127.0.0.1:59566] Received command: "cat error.txt"
[EXECUTING] [Client #1 - 127.0.0.1:59566] Executing command: "cat error.txt"
[OUTPUT] [Client #1 - 127.0.0.1:59566] Sending output to client:
ls: cannot access 'nonexistent_file': No such file or directory
[RECEIVED] [Client #2 - 127.0.0.1:17128] Received command: "echo "success" 2> er
ror2.txt"
[EXECUTING] [Client #2 - 127.0.0.1:17128] Executing command: "echo "success" 2>
error2.txt"
[OUTPUT] [Client #2 - 127.0.0.1:17128] Sending output to client:
success
[RECEIVED] [Client #2 - 127.0.0.1:17128] Received command: "cat error2.txt"
[EXECUTING] [Client #2 - 127.0.0.1:17128] Executing command: "cat error2.txt"
[OUTPUT] [Client #2 - 127.0.0.1:17128] Command executed successfully with no out
put.
$

khalaaasiyah — kag9691@DCLAP-V1525-CSD: ~/cshell — ssh kag9691@a...
-rw-rw-r-- 1 kag9691 kag9691 8508 Nov 7 09:40 client.c
-rw-rw-r-- 1 kag9691 kag9691 5504 Nov 7 09:45 client.o
-rw-rw-r-- 1 kag9691 kag9691 17977 Nov 7 09:40 executor.c
-rw-rw-r-- 1 kag9691 kag9691 5912 Nov 7 09:45 executor.o
-rw-rw-r-- 1 kag9691 kag9691 0 Nov 7 09:53 filelist.txt
-rw-rw-r-- 1 kag9691 kag9691 1785 Nov 7 09:40 input.c
-rw-rw-r-- 1 kag9691 kag9691 2621 Nov 7 09:40 main.c
-rw-rw-r-- 1 kag9691 kag9691 1802 Nov 7 09:42 Makefile
-rw-rw-r-- 1 kag9691 kag9691 2998 Nov 7 09:40 memory.c
-rw-rw-r-- 1 kag9691 kag9691 2016 Nov 7 09:45 memory.o
-rw-rw-r-- 1 kag9691 kag9691 41168 Nov 7 09:40 myshell
-rw-rw-r-- 1 kag9691 kag9691 12 Nov 7 09:52 output.txt
-rw-rw-r-- 1 kag9691 kag9691 14563 Nov 7 09:40 parser.c
-rw-rw-r-- 1 kag9691 kag9691 8216 Nov 7 09:45 parser.o
-rwxrwxr-x 1 kag9691 kag9691 35096 Nov 7 09:45 server
-rw-rw-r-- 1 kag9691 kag9691 18497 Nov 7 09:45 server.c
-rw-rw-r-- 1 kag9691 kag9691 11744 Nov 7 09:45 server.o
-rw-rw-r-- 1 kag9691 kag9691 1616 Nov 7 09:40 shell.h
-rw-rw-r-- 1 kag9691 kag9691 3309 Nov 7 09:40 utils.c
-rw-rw-r-- 1 kag9691 kag9691 2760 Nov 7 09:45 utils.o
$ ls nonexistent_file 2> error.txt
$ cat error.txt
ls: cannot access 'nonexistent_file': No such file or directory
$

khalaaasiyah — kag9691@DCLAP-V1525-CSD: ~/cshell — ssh kag9691@a...
Last login: Fri Nov 7 09:41:17 2025 from 10.228.235.67
kag9691@DCLAP-V1525-CSD:~$ cd cshell
kag9691@DCLAP-V1525-CSD:~/cshell$ ./client
Connected to remote shell server.
Type 'exit' to quit

$ pwd
/home/kag9691/cshell
$ echo one two three
one two three
$ echo "Test output" > output.txt
$ cat output.txt
Test output
$ echo -e "Line 1\nLine 2\nLine 3" > input.txt
$ cat < input.txt
Line 1
Line 2
Line 3
$ echo -e "zebra\napple\nbanana" > words.txt
$ sort < words.txt
apple
banana
zebra
$

khalaaasiyah — kag9691@DCLAP-V1525-CSD: ~/cshell — ssh kag9691@a...
total 240
-rwxrwxr-x 1 kag9691 kag9691 16936 Nov 7 09:45 client
-rw-rw-r-- 1 kag9691 kag9691 8508 Nov 7 09:40 client.c
-rw-rw-r-- 1 kag9691 kag9691 5504 Nov 7 09:45 client.o
-rw-rw-r-- 1 kag9691 kag9691 17977 Nov 7 09:40 executor.c
-rw-rw-r-- 1 kag9691 kag9691 5912 Nov 7 09:45 executor.o
-rw-rw-r-- 1 kag9691 kag9691 1785 Nov 7 09:40 input.c
-rw-rw-r-- 1 kag9691 kag9691 2621 Nov 7 09:40 main.c
-rw-rw-r-- 1 kag9691 kag9691 1802 Nov 7 09:42 Makefile
-rw-rw-r-- 1 kag9691 kag9691 2998 Nov 7 09:40 memory.c
-rw-rw-r-- 1 kag9691 kag9691 2016 Nov 7 09:45 memory.o
-rw-rw-r-- 1 kag9691 kag9691 41168 Nov 7 09:40 myshell
-rw-rw-r-- 1 kag9691 kag9691 14563 Nov 7 09:40 parser.c
-rw-rw-r-- 1 kag9691 kag9691 8216 Nov 7 09:45 parser.o
-rwxrwxr-x 1 kag9691 kag9691 35096 Nov 7 09:45 server
-rw-rw-r-- 1 kag9691 kag9691 18497 Nov 7 09:45 server.c
-rw-rw-r-- 1 kag9691 kag9691 11744 Nov 7 09:45 server.o
-rw-rw-r-- 1 kag9691 kag9691 1616 Nov 7 09:40 shell.h
-rw-rw-r-- 1 kag9691 kag9691 3309 Nov 7 09:40 utils.c
-rw-rw-r-- 1 kag9691 kag9691 2760 Nov 7 09:45 utils.o
$ echo "success" 2> error2.txt
success
$ cat error2.txt
success
$
```

Explanation: These tests check that error messages (stderr) can be redirected to files. Client 1's command generates an error, which is written to error.txt, while Client 2's command succeeds and writes nothing to error2.txt. Each client's error output is handled separately, showing that the server correctly manages stderr redirection for multiple clients.

5. Combined Redirections

Test Commands:

```
Client 3: $ echo -e "cat\ndog\nbird" > animals.txt
Client 3: $ sort < animals.txt > sorted_animals.txt
Client 3: $ cat sorted_animals.txt
Client 1: $ ls existing_file nonexistent_file > out.txt 2> err.txt
Client 1: $ cat out.txt
Client 2: $ cat err.txt
```

Screenshot 5: Combined Redirections

```
khalaasiyah -- kag9691@DCLAP-V1525-CSD: ~/cshell -- ssh kag9691@a...
[RECEIVED] [Client #3 - 127.0.0.1:12124] Received command: "cat sorted_animals.t
xt"
[EXECUTING] [Client #3 - 127.0.0.1:12124] Executing command: "cat sorted_animals
.txt"
[OUTPUT] [Client #3 - 127.0.0.1:12124] Sending output to client:
bird
cat
dog
[RECEIVED] [Client #1 - 127.0.0.1:59566] Received command: "ls existing_file non
existent_file > out.txt 2> err.txt"
[EXECUTING] [Client #1 - 127.0.0.1:59566] Executing command: "ls existing_file n
onexistent_file > out.txt 2> err.txt"
[OUTPUT] [Client #1 - 127.0.0.1:59566] Command executed successfully with no out
put.
[RECEIVED] [Client #1 - 127.0.0.1:59566] Received command: "cat out.txt"
[EXECUTING] [Client #1 - 127.0.0.1:59566] Executing command: "cat out.txt"
[OUTPUT] [Client #1 - 127.0.0.1:59566] Command executed successfully with no out
put.
[RECEIVED] [Client #2 - 127.0.0.1:17128] Received command: "cat err.txt"
[EXECUTING] [Client #2 - 127.0.0.1:17128] Executing command: "cat err.txt"
[OUTPUT] [Client #2 - 127.0.0.1:17128] Sending output to client:
ls: cannot access 'existing_file': No such file or directory
ls: cannot access 'nonexistent_file': No such file or directory
$

khalaasiyah -- kag9691@DCLAP-V1525-CSD: ~/cshell -- ssh kag9691@a...
-rw-rw-r-- 1 kag9691 kag9691 17977 Nov 7 09:40 executor.c
-rw-rw-r-- 1 kag9691 kag9691 5912 Nov 7 09:45 executor.o
-rw-rw-r-- 1 kag9691 kag9691 8 Nov 7 09:53 filelist.txt
-rw-rw-r-- 1 kag9691 kag9691 1785 Nov 7 09:40 input.c
-rw-rw-r-- 1 kag9691 kag9691 2621 Nov 7 09:40 main.c
-rw-rw-r-- 1 kag9691 kag9691 1802 Nov 7 09:42 Makefile
-rw-rw-r-- 1 kag9691 kag9691 2998 Nov 7 09:40 memory.c
-rw-rw-r-- 1 kag9691 kag9691 2016 Nov 7 09:45 memory.o
-rw-rw-r-- 1 kag9691 kag9691 41168 Nov 7 09:40 myshell
-rw-rw-r-- 1 kag9691 kag9691 12 Nov 7 09:52 output.txt
-rw-rw-r-- 1 kag9691 kag9691 14563 Nov 7 09:40 parser.c
-rw-rw-r-- 1 kag9691 kag9691 8216 Nov 7 09:45 parser.o
-rwxrwxr-x 1 kag9691 kag9691 35096 Nov 7 09:45 server
-rw-rw-r-- 1 kag9691 kag9691 18497 Nov 7 09:45 server.c
-rw-rw-r-- 1 kag9691 kag9691 11744 Nov 7 09:45 server.o
-rw-rw-r-- 1 kag9691 kag9691 1616 Nov 7 09:40 shell.h
-rw-rw-r-- 1 kag9691 kag9691 3309 Nov 7 09:40 utils.c
-rw-rw-r-- 1 kag9691 kag9691 2760 Nov 7 09:45 utils.o
$ ls nonexistent_file 2> error.txt
$ cat error.txt
ls: cannot access 'nonexistent_file': No such file or directory
$ ls existing_file nonexistent_file > out.txt 2> err.txt
$ cat out.txt
$

khalaasiyah -- kag9691@DCLAP-V1525-CSD: ~/cshell -- ssh kag9691@a...
$ pwd
/home/kag9691/cshell
$ echo one two three
one two three
$ echo "Test output" > output.txt
$ cat output.txt
Test output
$ echo -e "Line 1\nLine 2\nLine 3" > input.txt
$ cat < input.txt
Line 1
Line 2
Line 3
$ echo -e "zebra\napple\nbanana" > words.txt
$ sort < words.txt
apple
banana
zebra
$ echo -e "cat\ndog\nbird" > animals.txt
$ sort < animals.txt > sorted_animals.txt
$ cat sorted_animals.txt
bird
cat
dog
$

khalaasiyah -- kag9691@DCLAP-V1525-CSD: ~/cshell -- ssh kag9691@a...
-rw-rw-r-- 1 kag9691 kag9691 5504 Nov 7 09:45 client.o
-rw-rw-r-- 1 kag9691 kag9691 17977 Nov 7 09:40 executor.c
-rw-rw-r-- 1 kag9691 kag9691 5912 Nov 7 09:45 executor.o
-rw-rw-r-- 1 kag9691 kag9691 1785 Nov 7 09:40 input.c
-rw-rw-r-- 1 kag9691 kag9691 2621 Nov 7 09:40 main.c
-rw-rw-r-- 1 kag9691 kag9691 1802 Nov 7 09:42 Makefile
-rw-rw-r-- 1 kag9691 kag9691 2998 Nov 7 09:40 memory.c
-rw-rw-r-- 1 kag9691 kag9691 2016 Nov 7 09:45 memory.o
-rw-rw-r-- 1 kag9691 kag9691 41168 Nov 7 09:40 myshell
-rw-rw-r-- 1 kag9691 kag9691 14563 Nov 7 09:40 parser.c
-rw-rw-r-- 1 kag9691 kag9691 8216 Nov 7 09:45 parser.o
-rwxrwxr-x 1 kag9691 kag9691 35096 Nov 7 09:45 server
-rw-rw-r-- 1 kag9691 kag9691 18497 Nov 7 09:45 server.c
-rw-rw-r-- 1 kag9691 kag9691 11744 Nov 7 09:45 server.o
-rw-rw-r-- 1 kag9691 kag9691 1616 Nov 7 09:40 shell.h
-rw-rw-r-- 1 kag9691 kag9691 3309 Nov 7 09:40 utils.c
-rw-rw-r-- 1 kag9691 kag9691 2760 Nov 7 09:45 utils.o
$ echo "success" 2> error2.txt
success
$ cat error2.txt
$ cat err.txt
ls: cannot access 'existing_file': No such file or directory
ls: cannot access 'nonexistent_file': No such file or directory
$
```

Explanation: These tests verify that the server can handle multiple redirections at once. Input (<), output (>), and error (2>) redirections work correctly, even when several clients run commands simultaneously. Each client's files are separate, and outputs/errors are written to the correct files.

6. Pipes (|)

Test Commands:

Client 3: `$ echo "hello world" | wc -w`

Client 1: `$ ls | grep ".c"`

Client 2: `$ echo -e "apple\nbanana\napple\ncherry\napple" | sort | uniq -c`

Screenshot 6: Pipes (|)

```
khalaasiyah -- kag9691@DCLAP-V1525-CSD: ~/cshell -- ssh kag9691@a...
[EXECUTING] [Client #3 - 127.0.0.1:12124] Executing command: "echo "hello world"
| wc -w"
[OUTPUT] [Client #3 - 127.0.0.1:12124] Sending output to client:
2
[RECEIVED] [Client #1 - 127.0.0.1:59566] Received command: "ls | grep ".c""
[EXECUTING] [Client #1 - 127.0.0.1:59566] Executing command: "ls | grep ".c""
[OUTPUT] [Client #1 - 127.0.0.1:59566] Sending output to client:
client.c
executor.c
input.c
main.c
memory.c
parser.c
server.c
utils.c
[RECEIVED] [Client #2 - 127.0.0.1:17128] Received command: "echo -e "apple\nba
na\napple\ncherry\napple" | sort | uniq -c"
[EXECUTING] [Client #2 - 127.0.0.1:17128] Executing command: "echo -e "apple\nba
na\napple\ncherry\napple" | sort | uniq -c"
[OUTPUT] [Client #2 - 127.0.0.1:17128] Sending output to client:
3 apple
1 banana
1 cherry

khalaasiyah -- kag9691@DCLAP-V1525-CSD: ~/cshell -- ssh kag9691@a...
-rw-rw-r-- 1 kag9691 kag9691 14563 Nov 7 09:48 parser.c
-rw-rw-r-- 1 kag9691 kag9691 8216 Nov 7 09:45 parser.o
-rwxrwxr-x 1 kag9691 kag9691 35096 Nov 7 09:45 server
-rw-rw-r-- 1 kag9691 kag9691 18497 Nov 7 09:45 server.c
-rw-rw-r-- 1 kag9691 kag9691 11744 Nov 7 09:45 server.o
-rw-rw-r-- 1 kag9691 kag9691 1616 Nov 7 09:48 shell.h
-rw-rw-r-- 1 kag9691 kag9691 3309 Nov 7 09:48 utils.c
-rw-rw-r-- 1 kag9691 kag9691 2768 Nov 7 09:45 utils.o
$ ls nonexistent_file 2> error.txt
$ cat error.txt
ls: cannot access 'nonexistent_file': No such file or directory
$ ls existing_file nonexistent_file > out.txt 2> err.txt
$ cat out.txt
$ ls | grep ".c"
client.c
executor.c
input.c
main.c
memory.c
parser.c
server.c
utils.c
$

khalaasiyah -- kag9691@DCLAP-V1525-CSD: ~/cshell -- ssh kag9691@a...
$ echo one two three
one two three
$ echo "Test output" > output.txt
$ cat output.txt
Test output
$ echo -e "Line 1\nLine 2\nLine 3" > input.txt
$ cat < input.txt
Line 1
Line 2
Line 3
$ echo -e "zebra\napple\nbanana" > words.txt
$ sort < words.txt
apple
banana
zebra
$ echo -e "cat\ndog\nbird" > animals.txt
$ sort < animals.txt > sorted_animals.txt
$ cat sorted_animals.txt
bird
cat
dog
$ echo "hello world" | wc -w
2
$

khalaasiyah -- kag9691@DCLAP-V1525-CSD: ~/cshell -- ssh kag9691@a...
-rw-rw-r-- 1 kag9691 kag9691 2621 Nov 7 09:48 main.c
-rw-rw-r-- 1 kag9691 kag9691 1882 Nov 7 09:42 Makefile
-rw-rw-r-- 1 kag9691 kag9691 2998 Nov 7 09:48 memory.c
-rw-rw-r-- 1 kag9691 kag9691 2816 Nov 7 09:45 memory.o
-rw-rw-r-- 1 kag9691 kag9691 41168 Nov 7 09:48 myshell
-rw-rw-r-- 1 kag9691 kag9691 14563 Nov 7 09:48 parser.c
-rw-rw-r-- 1 kag9691 kag9691 8216 Nov 7 09:45 parser.o
-rwxrwxr-x 1 kag9691 kag9691 35096 Nov 7 09:45 server
-rw-rw-r-- 1 kag9691 kag9691 18497 Nov 7 09:45 server.c
-rw-rw-r-- 1 kag9691 kag9691 11744 Nov 7 09:45 server.o
-rw-rw-r-- 1 kag9691 kag9691 1616 Nov 7 09:48 shell.h
-rw-rw-r-- 1 kag9691 kag9691 3309 Nov 7 09:48 utils.c
-rw-rw-r-- 1 kag9691 kag9691 2768 Nov 7 09:45 utils.o
$ echo "success" 2> error2.txt
success
$ cat error2.txt
$ cat err.txt
ls: cannot access 'existing_file': No such file or directory
$ echo -e "apple\nbanana\napple\ncherry\napple" | sort | uniq -c
3 apple
1 banana
1 cherry
$
```

Explanation: These tests check that command pipelines work correctly, sending the output of one command as input to the next. Each client's pipeline is executed independently by its thread, and the server correctly manages multiple pipes without mixing outputs between clients.

7. Quotes Handling

Test Commands:

```
Client 3: $ echo 'Hello $USER'
Client 1: $ echo "Hello World"
Client 2: $ echo -e "Col1\tCol2\tCol3"
```

Screenshot 7: Quotes Handling

```
khalaasiyah — kag9691@DCLAP-V1525-CSD: ~/cshell — ssh kag9691@a...
[EXECUTING] [Client #3 - 127.0.0.1:12124] Executing command: "echo "hello world"
| wc -w"
[OUTPUT] [Client #3 - 127.0.0.1:12124] Sending output to client:
2
[RECEIVED] [Client #1 - 127.0.0.1:59566] Received command: "ls | grep ".c""
[EXECUTING] [Client #1 - 127.0.0.1:59566] Executing command: "ls | grep ".c""
[OUTPUT] [Client #1 - 127.0.0.1:59566] Sending output to client:
client.c
executor.c
executor.o
input.c
main.c
memory.c
parser.c
server.c
utils.c
[RECEIVED] [Client #2 - 127.0.0.1:17128] Received command: "echo -e "apple\banana\napple\ncherry\napple" | sort | uniq -c"
[EXECUTING] [Client #2 - 127.0.0.1:17128] Executing command: "echo -e "apple\banana\napple\ncherry\napple" | sort | uniq -c"
[OUTPUT] [Client #2 - 127.0.0.1:17128] Sending output to client:
3 apple
1 banana
1 cherry

$ echo "Test output" > output.txt
$ cat output.txt
Test output
$ echo -e "Line 1\nLine 2\nLine 3" > input.txt
$ cat < input.txt
Line 1
Line 2
Line 3
$ echo -e "zebra\napple\nbanana" > words.txt
$ sort < words.txt
apple
banana
zebra
$ echo -e "cat\ndog\nbird" > animals.txt
$ sort < animals.txt > sorted_animals.txt
$ cat sorted_animals.txt
bird
cat
dog
$ echo "hello world" | wc -w
2
$ echo 'Hello $USER\'
Hello $USER\
$

khalaasiyah — kag9691@DCLAP-V1525-CSD: ~/cshell — ssh kag9691@a...
-rwxrwxr-x 1 kag9691 kag9691 35896 Nov 7 09:45 server
-rw-rw-r-- 1 kag9691 kag9691 18497 Nov 7 09:45 server.c
-rw-rw-r-- 1 kag9691 kag9691 11744 Nov 7 09:45 server.o
-rw-rw-r-- 1 kag9691 kag9691 1616 Nov 7 09:40 shell.h
-rw-rw-r-- 1 kag9691 kag9691 3389 Nov 7 09:40 utils.c
-rw-rw-r-- 1 kag9691 kag9691 2760 Nov 7 09:45 utils.o
$ ls nonexistent_file 2> error.txt
$ cat error.txt
ls: cannot access 'nonexistent_file': No such file or directory
$ ls existing_file nonexistent_file > out.txt 2> err.txt
$ cat out.txt
$ ls | grep ".c"
client.c
executor.c
executor.o
input.c
main.c
memory.c
parser.c
server.c
utils.c
$ echo "Hello World"
Hello World
$

khalaasiyah — kag9691@DCLAP-V1525-CSD: ~/cshell — ssh kag9691@a...
-rw-rw-r-- 1 kag9691 kag9691 2998 Nov 7 09:40 memory.c
-rw-rw-r-- 1 kag9691 kag9691 2816 Nov 7 09:45 memory.o
-rw-rw-r-- 1 kag9691 kag9691 41168 Nov 7 09:40 myshell
-rw-rw-r-- 1 kag9691 kag9691 14563 Nov 7 09:40 parser.c
-rw-rw-r-- 1 kag9691 kag9691 8216 Nov 7 09:45 parser.o
-rwxrwxr-x 1 kag9691 kag9691 35896 Nov 7 09:45 server
-rw-rw-r-- 1 kag9691 kag9691 18497 Nov 7 09:45 server.c
-rw-rw-r-- 1 kag9691 kag9691 11744 Nov 7 09:45 server.o
-rw-rw-r-- 1 kag9691 kag9691 1616 Nov 7 09:40 shell.h
-rw-rw-r-- 1 kag9691 kag9691 3389 Nov 7 09:40 utils.c
-rw-rw-r-- 1 kag9691 kag9691 2760 Nov 7 09:45 utils.o
$ echo "success" 2> error2.txt
success
$ cat error2.txt
$ cat err.txt
ls: cannot access 'existing_file': No such file or directory
ls: cannot access 'nonexistent_file': No such file or directory
$ echo -e "apple\nbanana\napple\ncherry\napple" | sort | uniq -c
3 apple
1 banana
1 cherry
$ echo -e "Col1\tCol2\tCol3"
Col1 Col2 Col3
$
```

Explanation: These tests verify that the server correctly handles single and double quotes:

- Single quotes preserve text literally ('Hello \$USER' prints exactly as typed).
 - Double quotes allow spaces and escape sequences ("Hello World", \t tab).
- Each client's thread processes quoted text independently, ensuring commands are interpreted correctly without interfering with other clients.

8. Complex Pipelines with Redirections

Test Command:

Client 3: \$ ls -l | grep ".c" > c_files.txt

Client 3: \$ cat c_files.txt

Client 1: \$ echo -e "dog\ncat\nbird" > animals2.txt

Client 1: \$ cat < animals2.txt | sort | uniq > unique_animals.txt 2> errors.txt

Client 1: \$ cat unique_animals.txt

Screenshot 8: Complex Pipelines with Redirections

```
[EXECUTING] [Client #3 - 127.0.0.1:12124] Executing command: "echo "hello world"
| wc -w"
[OUTPUT] [Client #3 - 127.0.0.1:12124] Sending output to client:
2
[RECEIVED] [Client #1 - 127.0.0.1:59566] Received command: "ls | grep ".c""
[EXECUTING] [Client #1 - 127.0.0.1:59566] Executing command: "ls | grep ".c""
[OUTPUT] [Client #1 - 127.0.0.1:59566] Sending output to client:
client.c
executor.c
executor.o
input.c
main.c
memory.c
parser.c
server.c
utils.c
[RECEIVED] [Client #2 - 127.0.0.1:17128] Received command: "echo -e "apple\\nbana
na\\napple\\ncherry\\napple" | sort | uniq -c"
[EXECUTING] [Client #2 - 127.0.0.1:17128] Executing command: "echo -e "apple\\nba
nana\\napple\\ncherry\\napple" | sort | uniq -c"
[OUTPUT] [Client #2 - 127.0.0.1:17128] Sending output to client:
3 apple
1 banana
1 cherry

$ sort < animals.txt > sorted_animals.txt
$ cat sorted_animals.txt
bird
cat
dog
$ echo "hello world" | wc -w
2
$ echo 'Hello $USER\'
Hello $USER\
$ ls -l | grep ".c" > c_files.txt
$ cat c_files.txt
-rw-r--r-- 1 kag9691 kag9691 0 Nov 7 10:03 c_files.txt
-rwxrwxr-x 1 kag9691 kag9691 16936 Nov 7 09:45 client
-rw-rw-r-- 1 kag9691 kag9691 8508 Nov 7 09:40 client.c
-rw-rw-r-- 1 kag9691 kag9691 5504 Nov 7 09:45 client.o
-rw-rw-r-- 1 kag9691 kag9691 17977 Nov 7 09:40 executor.c
-rw-rw-r-- 1 kag9691 kag9691 5912 Nov 7 09:45 executor.o
-rw-rw-r-- 1 kag9691 kag9691 1785 Nov 7 09:40 input.c
-rw-rw-r-- 1 kag9691 kag9691 2621 Nov 7 09:40 main.c
-rw-rw-r-- 1 kag9691 kag9691 2998 Nov 7 09:40 memory.c
-rw-rw-r-- 1 kag9691 kag9691 14563 Nov 7 09:40 parser.c
-rw-rw-r-- 1 kag9691 kag9691 18497 Nov 7 09:45 server.c
-rw-rw-r-- 1 kag9691 kag9691 3309 Nov 7 09:40 utils.c

$

-rw-rw-r-- 1 kag9691 kag9691 2760 Nov 7 09:45 utils.o
$ ls nonexistent_file 2> error.txt
$ cat error.txt
ls: cannot access 'nonexistent_file': No such file or directory
$ ls existing_file nonexistent_file > out.txt 2> err.txt
$ cat out.txt
$ ls | grep ".c"
client.c
executor.c
executor.o
input.c
main.c
memory.c
parser.c
server.c
utils.c
$ echo "Hello World"
Hello World
$ echo -e "dog\ncat\nbird" > animals2.txt
$ cat < animals2.txt | sort | uniq > unique_animals.txt 2> errors.txt
$ cat unique_animals.txt
bird
cat
dog

-rw-rw-r-- 1 kag9691 kag9691 2998 Nov 7 09:40 memory.c
-rw-rw-r-- 1 kag9691 kag9691 2016 Nov 7 09:45 memory.o
-rw-rw-r-- 1 kag9691 kag9691 41168 Nov 7 09:40 myshell
-rw-rw-r-- 1 kag9691 kag9691 14563 Nov 7 09:40 parser.c
-rw-rw-r-- 1 kag9691 kag9691 8216 Nov 7 09:45 parser.o
-rwxrwxr-x 1 kag9691 kag9691 35096 Nov 7 09:45 server
-rw-rw-r-- 1 kag9691 kag9691 18497 Nov 7 09:45 server.c
-rw-rw-r-- 1 kag9691 kag9691 11744 Nov 7 09:45 server.o
-rw-rw-r-- 1 kag9691 kag9691 1616 Nov 7 09:40 shell.h
-rw-rw-r-- 1 kag9691 kag9691 3309 Nov 7 09:40 utils.c
-rw-rw-r-- 1 kag9691 kag9691 2760 Nov 7 09:45 utils.o
$ echo "success" 2> error2.txt
success
$ cat error2.txt
$ cat err.txt
ls: cannot access 'existing_file': No such file or directory
ls: cannot access 'nonexistent_file': No such file or directory
$ echo -e "apple\\nbana\\napple\\ncherry\\napple" | sort | uniq -c
3 apple
1 banana
1 cherry
$ echo -e "Col1\tCol2\tCol3"
Col1 Col2 Col3
$
```

Explanation: These tests check that the server can handle pipelines combined with input, output, and error redirection. Each client thread executes its complex command independently:

- Pipelines pass output from one command to the next.
- Input redirection (<) reads from a file, output (>) writes to a file, and error (2>) writes errors separately.
- The server ensures that multiple clients can run such commands at the same time without interfering with each other.

Challenges

Challenge	Description	Resolution
Race conditions in log output	Multiple threads printing simultaneously caused garbled text	Introduced <code>output_mutex</code> and <code>thread_safe_printf()</code>
Duplicate client IDs	Concurrent threads incrementing counter simultaneously	Protected counter with <code>counter_mutex</code>
Memory leaks	Allocated <code>ClientInfo</code> structs not freed	Each thread frees its own <code>ClientInfo</code> before exiting
Thread resource accumulation	Threads remained joinable	Used <code>pthread_detach()</code> for automatic cleanup

Division of Tasks

This project was completed individually.
I implemented all aspects of:

- Threaded server architecture.
- Synchronization logic (mutexes and thread-safe logging).
- Client handling and cleanup.
- Integration with existing shell core logic.
- Testing and debugging.

References

1. Phase 2 Implementation Report – Garrett, Khaleeqa Aasiyah (2025)
2. Stevens, W. Richard, and Rago, Stephen A. *Advanced Programming in the UNIX Environment*, 3rd ed., Addison-Wesley, 2013.
3. Kerrisk, Michael. *The Linux Programming Interface*. No Starch Press, 2010.
4. Beej's Guide to Network Programming.
5. LLNL Pthreads Programming Tutorial.
6. Linux man pages for `pthread_create`, `mutex`, `accept`, `dup2`, `fork`, `pipe`.